Operating System Notes

Chapter 4-6

程序-进程-线程

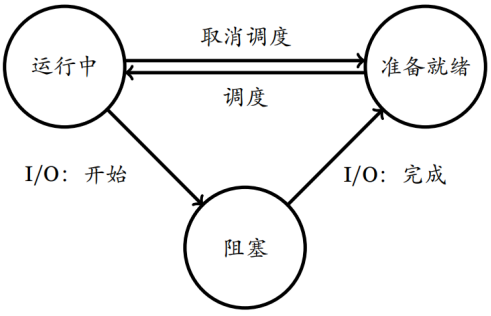
为了处理并发程序，操作系统必须分离不同程序的执行，为程序提供每个程序都是唯一正在运行的程序的幻觉/假象。**虚拟进程抽象提供了这种幻觉**

一个进程是在进程状态上下文中的一个执行流。执行流是执行指令的序列（即“控制线程”）。进程状态涵盖了执行指令可能影响或受到影响的所有内容（例如，寄存器、地址空间、持久状态，如文件）。

类型	定义	区别
程序	存储在磁盘上的应用程序	包含磁盘上的可执行文件
进程	程序的一个运行实例	具有初始化的数据段和堆栈
线程	由进程启动，可创建多个	共享全局数据、代码和堆

共享资源：时间共享（逐个运行）/ 空间共享（平均分配）

进程的五种状态（运行中、准备就绪、阻塞、新、死亡）



如果所有进程都被阻塞时应该调用空闲进程（不会阻塞任何I/O），其保证了始终有一个进程可以运行

进程数据结构：

```
进程标识符（PID）
进程状态（例如，准备就绪）
指向父进程的指针（cat /proc/self/status）
CPU 上下文（如果进程未运行）
地址空间指针（cat /proc/self/maps）
打开文件列表的指针（文件描述符，cat /proc/self/fdinfo/*）
```

进程API

> 创建：在 UNIX 中，fork() 系统调用创建进程。
>
> 终止：在 UNIX 中，exit() 系统调用终止进程。
>
> 其他功能：等待进程（wait()/waitpid()）、自愿阻塞（sleep()）、替
>
> 换程序（exec()）、信号处理（signal()）、时间设置（setitimer()）

Fork：复制现有进程

父进程调用 fork() 系统调用。

内核（操作系统）创建一个新的进程，称为子进程。它与父进程几乎完全相同。

父进程和子进程都从 fork() 调用返回。在父进程中，返回子进程的 PID；在子进程中，返回 0。

操作系统安排父进程和子进程交替执行。

exec()，执行（新）程序

总是执行相同的程序很无聊（我们需要一个包含所有功能的大型程序，例如 emacs）。

exec() 替换地址空间，从磁盘加载新程序。

程序可以传递命令行参数和环境。

旧的地址空间/状态被销毁，除了 STDIN、STDOUT、STDERR 保留外，允许父进程重定向/重新连接子进程的输出！

wait()，等待子进程

子进程与其父进程绑定。

exit(int retval) 接受一个返回值参数。

父进程可以 wait() 等待子进程的终止并读取子进程的返回值。

进程树

每个进程都有一个父进程

一个进程可以有多个子进程

每个进程又可以有子进程

Chapter 7-9

对于进程来说，上下文切换是透明的（也就是说进程是无感的）

在分析调度策略时，我们使用以下术语：

- 利用率：CPU 执行程序的时间占总时间的比例（目标：最大化）
- 周转时间：从任务完成时间减去任务到达系统的时间（目标：最小化）
- 响应时间：从到达系统到被调度的时间（目标：最小化）
- 公平性：所有进程在一段时间内获得相同数量的 CPU （目标：无饥饿）
- 进展：允许进程向前推进（目标：最小化内核中断）

先前的调度器（FIFO、SJF）是非抢占式的。非抢占式调度器只有在当前进程自愿放弃 CPU 时才会切换到另一个进程。

抢占式调度器可以在任何时候获取 CPU 控制权，根据调度策略切换到另一个进程。

- 批处理进程：响应时间不重要，关注长时间运行（减少上下文切换成本，关注大量 CPU，不关注时间）
- 交互式进程：响应时间至关重要，短暂突发（上下文切换成本不重要，不需要太多 CPU 但频繁）

MLFQ 总结

- 规则 1：如果 prio(A) > prio(B)，则运行 A。
- 规则 2：如果 prio(A) == prio(B)，则 A、B 轮流运行
- 规则 3：新进程从顶部优先级开始
- 规则 4：当 (某一层) 整个时间片被使用时，降低进程的优先级
- 规则 5：定期将所有作业移动到最顶部的队列

CFS：完全公平调度器

思想：每个任务并行运行，并消耗相等的 CPU 时间份额

方法：计算进程在理想处理器上获得的时间

示例：假设有 4 个进程准备就绪，因此它们每个将获得 CPU 的 1/4（将此时间添加到记账中）

在真实硬件上，一次只能运行一个任务，因此 CFS 引入“虚拟运行时间”的概念。任务的虚拟运行时间指定了其下一个时间片将开始执行的时间。实际上，任务的虚拟运行时间是基于运行的任务总数对实际运行时间的归一化操作。

- CFS 跟踪每个进程在理想处理器上应该执行的时间。
- 对于每个时间片，它计算每个进程将接收到的分数，并将这些值保留在红黑树中。
- 然后调度具有最高值的进程
- Linux 使用基于多级反馈队列的 O(1) 调度器，但在 2007 年切换到了完全公平调度器

实现：将所有进程保留在红黑树中，按最大执行时间排序（跟踪其正余额）

调度

1. 调度最左边的进程（具有最高余额的进程）
2. 如果进程退出，则将其从调度树中移除
3. 在中断（时间片结束或 I/O）时，将进程重新插入树中的新位置
4. 重复

上下文切换和抢占是允许操作系统保持控制并实现更高级调度策略的基本机制。

调度器需要优化不同的指标：利用率、周转时间、响应时间、公平性和前进度

- FIFO：简单、非抢占调度器
- SJF：非抢占，防止进程堵塞
- STFC：抢占，防止后期进程堵塞
- RR：抢占，响应时间好，周转时间差
- MLFQ：抢占，最为现实
- CFS：通过虚拟化时间实现公平调度

基本原则：过去行为是未来行为的良好预测器

Chapter 13-17

虚拟化的目标：将进程（及其故障）彼此隔离。

多道程序设计的目标：

- 透明性：进程不用关心其他进程的存在
- 保护：OS/其他进程与一个进程内存上相互不影响
- 效率：不浪费资源（如碎片化）、尽可能地接近硬件性能
- 共享：进程可能共享部分地址空间

地址空间：每个进程都有一组地址，映射到数据（即，从指针到字节的映射）

- 静态：代码和全局变量
- 动态：栈、堆

为什么需要动态内存?

- 所需内存量不确定
- 输入大小编译时未知
- 保守的分配方法是浪费的

函数返回时数据会发生什么变化?

- 前一个函数的数据，在下一个函数初始化其数据时会被覆盖

```
int a = 2;
int called(int b) {
  int c = a * b;
  printf("a: %d b: %d c: %d\n", a, b, c);
  a = 5;
  return c;
}
int main(int argc, char* argv) {
  int b = 2, c = 3;
  printf("a: %d b: %d c: %d\n", a, b, c);
  b = called(c);
  printf("a: %d b: %d c: %d\n", a, b, c);
  return 0;
}
```

a: 2 b: 2 c: 3 || a: 2 b: 3 c: 6 || a: 5 b: 6 c: 3

堆的分配：找到一个合适的对象（首次适配、最佳适配、最差适配）

- 首次适配：找到列表中的第一个对象并分割它
- 最佳适配：找到大小最接近的对象
- 最差适配：找到最大的对象并分割它

堆的释放：合并相邻的块

- 如果相邻区域为空闲，则合并这两个块

虚拟化内存：时间共享还是空间共享?

如何防止进程修改 MMU 配置?

- 分离：操作系统以高于进程的权限运行
- 操作系统权限包括用于 MMU 配置的特殊指令
- 通过系统调用（特殊的调用指令）从用户空间（进程）切换到内核空间
- 操作系统再返回到不受特权限制的用户模式
- 用户空间中的异常（例如，非法内存访问、除零）切换到特权模式，操作系统处理异常

- 这个设计是否没有安全问题?
  - P1 和 P2 真的分离吗?

否！P1 可以访问 P2 的内存，因为基址寄存器只是简单地添加偏移量。在前面的示例中，使用 `base=0x1000`，访问地址 $0x2000_v$ 将在 P1 执行时访问 P2 的内存的第一个字节！

简单解决方案：基址和边界

- 基址寄存器设置最小地址
- 边界寄存器设置地址空间（虚拟）的限制，最高可访问的物理地址为base+bounds

Chapter 18-22

地址空间封装了所有可寻址的内存，假设每个地址指向**一个字节**

内存抽象的七个层次：

1. 无虚拟化，程序在裸金属上运行
2. 分时共享，一次只运行一个程序
3. 空间共享，在一个共享的地址空间中重新定位程序
4. 空间共享，通过段（基址）实现虚拟地址空间
5. 空间共享，通过段（基址 + 边界）实现虚拟地址空间
6. 空间共享，通过多个段实现虚拟地址空间
7. 空间共享，通过分页实现虚拟地址空间

分段允许在多个进程之间合理共享物理地址空间。有什么缺点呢?

- 空间：每个段必须完全由内存支持
- 空间：物理内存区域必须是连续的（碎片化）

- ISA：指令明确或隐含地编码目标段

碎片化：无法使用的内存。

- 外部碎片化：对分配器（操作系统）可见，例如，段之间。free后的内存没有合并。
- 内部碎片化：对请求者可见，例如，如果段大小是 *2* 的幂，则舍入。分配内存的时因为最小片段的问题造成碎片。

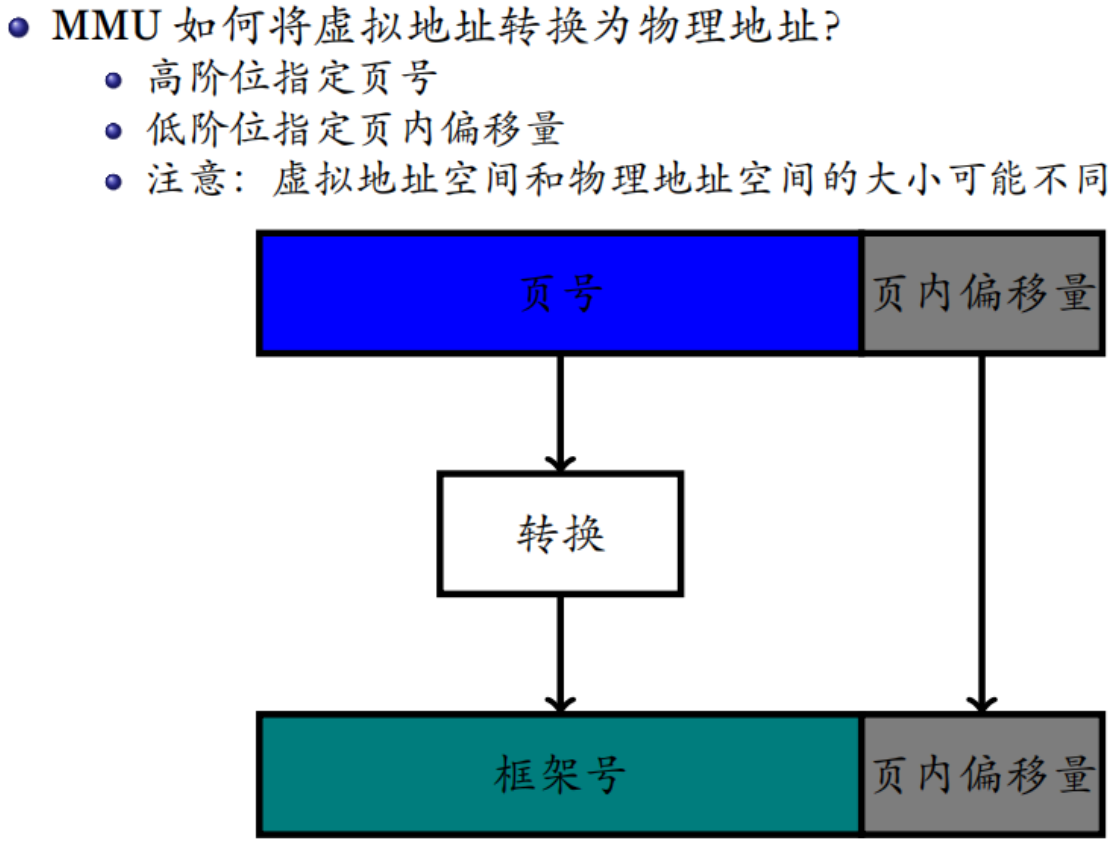
页面是分割地址空间的最小单位

对于进程（虚拟地址空间），这称为（虚拟）页面

作为物理地址空间的一部分，这称为框架（或物理页面）

具有 12 位页面的系统意味着地址的最低 12 位指定页面偏移量

页面的大小是 2^12 字节（4 KiB）



- 虚拟地址空间：4 GiB（即 32 位）（2^2 * 2^30）TB->GB->MB->KB->B(进制1024)
- 物理地址空间：256 TiB（即 48 位）
- 页面大小：4 KiB（即 12 位）
- 页面号使用 20 位（即每个进程可能引用 2^20 个 4 KiB 大小的页面）
- 框架号使用 36 位（系统中最多有 2^36 个物理 4 KiB 框架）

假设虚拟地址有 16 位，物理地址空间有 20 位，页面大小为 12 位。

对于虚拟地址，offset有4位，对应每个进程的页数16页；

对于物理地址，offset有8位，对应物理页面可以有256个

这意味着每个进程有 16 页，物理页面有 256 个。

分页

优点

- 无外部碎片
- 分配速度快（无需搜索空间）和释放速度快（无需合并）
- 简单调整映射到内核的子集（稍后）

缺点

- 需要额外的内存引用到页表（提示：使用高速缓存）
- 内部碎片（平均为页面大小的一半）
- 页表所需的空间可能很大

TLB

假设 12 位用于页，32 位虚拟和 32 位物理地址空间，每个页面表项为 4 字节。

单级页表的大小是多少?

12位用于页表示Page Offset = 12位，则VPN = 32 - 12 = 20 位

页表大小=页表个数*页表单个大小

页表个数=2^(VPN) = 2^20

页表单个大小=4B

页表大小=2^22字节 = 4MB

假设 32 位虚拟和 48 位物理地址空间以及 8 字节条目：8 MiB

假设 64 位虚拟地址空间和 8 字节条目：2^52 $*$ 8B（32 PiB）。**页表容量过大!**

大多数进程只需要一小部分地址空间。

- 各个级别的大小应该是多少?
- 想法：为 32 位虚拟/物理地址使用页面粒度!
  - 4 KiB 页面：1024 x 4 B 条目
  - 1024 个条目对应 10 位
  - 将地址分成 10 位第一级、10 位第二级、12 位偏移：0x3ff 0x3ff 0xfff

第一级	第二级	偏移

！！32位系统的一个PTE是4字节，而64位系统的一个PTE（页表条目大小）是8字节！！

让我们转移到 64 位虚拟/物理地址空间，4 KiB 页面

4 KiB 页面有空间容纳512 个条目（9 位）（4*1024 / 8 = 512 = 2^9）也就是一个页表需要9位来表示所有的页面

64-12 =52，也就是需要6级页表（52/9）

- 每次内存访问都会进行虚拟到物理的转换
- MMU 在页面错误期间为什么切换到操作系统?

- 操作系统处理**策略**，MMU 是**机制**
- MMU 处理常见情况：页面路径有效，页面存在
  - 页面遍历可以高度优化
  - 操作系统和 MMU 对数据结构达成一致
- 操作系统处理策略决策
  - 允许实现灵活的策略
  - 操作系统决定哪些页面被分配和替换
  - 操作系统也可以将信息传递给进程，例如，如果发生非法访问
  - 由于页面错误很少，因此不需要极高的速度

程序的工作集被定义为程序代码和数据的子集，这些代码和数据当前正在使用。这意味着工作集是一个随着程序的运行而随时间变化的临时指标。大多数访问将被限制在少量地址上，表现出局部性。

工作集 (workset)

- 访存的局部性：在进程运行的任何阶段，它都只访问它的页面中较小的一部分
- 工作集：一个进程当前使用的页的集合
- 抖动：分配给进程的物理页面数太小，无法包含其工作集，频繁地在内存和外存间换页
- 缺页率：一秒钟内出现的缺页数 (统计)
- 负载控制：系统内运行的进程过多，无法使所有进程的缺页率都低于 A，则需要将一些进程换出至外存 (交换)

CPU 如何执行读/写操作?

- CPU 发出虚拟地址的加载（作为内存加载/存储或已执行指令的一部分）
- MMU 检查 TLB 中的虚拟地址
  - TLB 未命中：MMU 执行页面遍历
    - 页表条目存在：更新 TLB，继续
    - 不在但有效：页面错误，切换到操作系统，操作系统映射页面并透明地返回到进程
    - 无效：页面错误，切换到操作系统，操作系统引发 SEG 错误并将其传递给程序
  - TLB 命中：获取物理地址，获取内存位置并返回到 CPU

Chapter 28-30

线程是由进程创建的，独立的执行上下文，且能共享相同的地址空间（详见Chapter4-6）

- **并行性**：多个线程（或进程）在多个 CPU 内核上执行单个任务
- **并发性**：任务可以在重叠的时间段内启动、运行和完成，例如通过交叉执行它们的执行，或者通过并行执行它们时重叠它们的执行

请注意，进程可以通过部分重叠的地址空间或通过进程间通信共享信息。

这意味着如果会产生竞态条件，如果不加以（Lock）限制就会出现操作丢失等等很严重的问题

竞态条件

并发执行导致竞争条件，所以访问共享数据必须被管控

- **临界区**：访问共享数据的代码部分
- **互斥**：任何时刻只允许一个进程执行临界区
- **原子性**：临界区作为不可中断块执行

上锁是实现原子性一个机制。

锁

锁保护临界区的变量，所有竞争同一个临界区的进程必须共享同一个锁，同时刻只有一个线程能获取到锁，否则就要wait

锁的要求

1. 互斥：临界区只能有一个线程
2. 公平：所有线程最终应获得锁
3. 性能：获取/释放锁的开销低

自旋锁

工作原理： 当一个线程尝试获取自旋锁但锁已被其他线程持有时，它不会被挂起（进入睡眠状态），而是会**忙等待**（"自旋"），不断地循环检查锁是否可用。它会反复地尝试获取锁，直到成功为止。

优点：

- **避免上下文切换开销：** 由于线程不休眠，所以避免了操作系统进行线程上下文切换的开销。上下文切换是比较耗时的操作。
- **响应速度快：** 如果锁被持有的时间很短，自旋锁可以非常快地获取到锁，因为不需要进行昂贵的线程调度操作。

缺点：

- **CPU 浪费：** 如果锁被持有的时间较长，或者竞争激烈，自旋等待的线程会白白占用 CPU 资源，进行无意义的循环检查，造成 CPU 浪费。
- **不公平性：** 自旋锁通常不保证公平性，即哪个线程先尝试获取锁不一定能先获取到锁，可能会出现“饥饿”现象。
- **单核系统效率低：** 在单核处理器系统中，自旋锁的缺点更为突出。如果持有锁的线程被中断并切换出去，而另一个自旋的线程被调度上来，那么这个自旋的线程会一直空转，直到持有锁的线程再次被调度并释放锁。

队列锁

工作原理： 当一个线程尝试获取队列锁但锁已被占用时，它不会立即忙等待，而是会将自己**排入一个等待队列**。当持有锁的线程释放锁时，它会通知队列中的下一个等待线程来获取锁。

优点：

- **减少 CPU 浪费：** 避免了线程的忙等待，减少了 CPU 资源的浪费，尤其是在锁竞争激烈的情况下。

- **公平性：** 通常能够提供更强的公平性，按照排队的顺序来获取锁，避免了“饥饿”现象。
- **适用于锁持有时间较长的场景：** 当锁可能被长时间持有或者竞争激烈时，队列锁的性能优于普通自旋锁。

缺点：

- **上下文切换开销：** 线程被放入等待队列和从队列中唤醒都需要操作系统的上下文切换，这会带来一定的开销。
- **实现更复杂：** 相比简单的自旋锁，队列锁的实现更为复杂，需要维护一个等待队列。

自旋锁与队列锁的区别

- **当临界区短暂而稀疏时，自旋锁效果很好，而且我们在多个 CPU 上执行（没有上下文切换，很可能很快就能获得锁）**
- **当临界区较长或频繁时，队列锁效果很好（高争用情况，线程必须等待的可能性较大）**

Chapter 30-32

条件变量 (Condition Variables, CV)

我们先从一个常见场景开始：一个线程需要等待另一个线程完成某个任务。

问题：空转 (Spinning) 是巨大的浪费

一个最直观的想法是使用一个全局标志位，比如 `done`。子线程完成任务后就设置 `done = true`。而父线程则在一个循环里不停地检查这个标志位：`while (!done);`。

这种方式叫做**空转**或**忙等待 (busy-waiting)**。它最大的问题就是**效率极低**。父线程在等待的期间，并没有去做任何有意义的工作，却一直在消耗CPU时间片。这就像你在等微波炉热饭，不是设定好时间去做别的事，而是一直死死地盯着微波炉的秒表，这纯粹是在浪费你自己的时间（CPU资源）。

解决方案：用条件变量进行高效等待

为了解决空转的浪费，我们引入了**条件变量 (CV)**。你可以把它理解成一个“等待室”或者一个线程队列。当一个线程发现它需要的条件（例如，“任务已完成”）还不满足时，它就可以调用 `wait` 操作，把自己放进这个等待室里睡觉，主动让出CPU。然后，当另一个线程完成了任务，满足了这个条件时，它就调用 `signal` 操作，去等待室里唤醒一个正在沉睡的线程。

核心API：

- `wait`：使当前线程进入等待（睡眠）状态，直到被唤醒。
- `signal`：唤醒**一个**正在等待该条件的线程。
- `broadcast`：唤醒**所有**正在等待该条件的线程。

正确使用条件变量 (The Golden Rules)

使用条件变量有几个必须遵守的铁律，我们来看一个正确的 `thr_join` 实现：

```
bool done = false;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER; // 互斥锁
pthread_cond_t c = PTHREAD_COND_INITIALIZER;  // 条件变量

void thr_exit() { // 子线程调用
    pthread_mutex_lock(&m);
```

```
    done = true;
    pthread_cond_signal(&c); // 唤醒等待者
    pthread_mutex_unlock(&m);
}

void thr_join() { // 父线程调用
    pthread_mutex_lock(&m);
    while (!done) {
        pthread_cond_wait(&c, &m); // 等待
    }
    pthread_mutex_unlock(&m);
}
```

这里有三个非常关键的知识点：

1. 为什么必须配合互斥锁 (Mutex) 使用？

   想象一下如果没有锁会发生什么，这会产生一种经典的竞态条件，叫做“错失的唤醒 (Lost Wakeup)”：

   - 父线程检查 `done`，发现是 `false`。
   - 此时，操作系统中断了父线程，切换到子线程执行。
   - 子线程将 `done` 设为 `true`，然后调用 `pthread_cond_signal`。但此时父线程还没来得及睡下，**所以这个 `signal` 信号就丢失了，没有人接收。**
   - 操作系统切换回父线程，它从刚刚中断的地方继续执行，调用 `pthread_cond_wait` 开始睡眠，结果可能永远也醒不过来了。 **互斥锁 `m` 就是为了保证“检查条件”和“进入睡眠”这两个动作打包成一个原子操作**，杜绝中途被打断的可能。

2. pthread_cond_wait(&c, &m) 的内部魔法

   这个函数的操作是原子性的，它内部做了三件事：

   - 释放传入的互斥锁 `m`。
   - 将线程自身放入条件变量 `c` 的等待队列中（进入睡眠）。
   - 当被唤醒后，它会**重新自动获取**互斥锁 `m`，然后才返回。

3. 为什么必须用 while 而不是 if 来检查条件？

   这是初学者最容易犯的错误。使用 while 是为了处理两种情况：

   - **多个等待者**：如果多个父线程都在等待一个子线程，当子线程发出 `signal` 时，只有一个等待者会被唤醒。但如果它用的是 `broadcast`，所有等待者都会被唤醒并争抢锁。第一个抢到锁的线程会发现 `done` 为 `true`，处理完后释放锁。但其他随后抢到锁的线程如果用的是 `if`，它们会误以为条件依然满足而直接往下走，这是错误的。 `while` 循环能确保每个被唤醒的线程都**重新检查 (re-check)** 条件，确保万无一失。
   - **虚假唤醒 (Spurious Wakeups)**：在某些操作系统实现中，线程可能会在没有 `signal` 信号的情况下被“意外”唤醒。虽然不常见，但 `while` 循环可以优雅地处理这种情况，确保程序的健壮性。

信号量 (Semaphores)

接下来我们看信号量。你可以把信号量看作一个更广义的同步工具。它本质上是一个**整数计数器**，并配有两个原子操作。

- `sem_wait(sem_t *s)`：这个操作会先检查信号量的值。**如果值大于0，就将其减1并立即返回。**如果值等于0，线程就会进入**睡眠**，直到有其他线程对这个信号量执行 `post` 操作。这个操作在学术上常被称为 `P()` 或 `down()`。
- `sem_post(sem_t *s)`：这个操作将信号量的值加1。如果此时有线程因为这个信号量正在睡眠，`post` 操作会唤醒其中一个。这个操作常被称为 `V()` 或 `up()`。

信号量的两种用法

1. **二元信号量 (Binary Semaphore)**：将信号量的值初始化为1，它就变成了一个互斥锁。`sem_wait()` 就是加锁，`sem_post()` 就是解锁。
2. **计数信号量 (Counting Semaphore)**：这是信号量最强大的地方。它可以用来管理一组有限的资源。例如，一个资源池里有N个资源，就可以把信号量初始化为N。每个想使用资源的线程都必须先 `sem_wait()`，用完后再 `sem_post()`。

经典应用：生产者-消费者问题

这是并发编程中最经典的问题之一。想象一个固定大小的共享缓冲区，生产者向里面放东西，消费者从里面取东西。我们需要保证：

- 生产者不能在缓冲区满的时候继续放东西。
- 消费者不能在缓冲区空的时候去取东西。

我们可以用两个信号量完美地解决这个问题：

- `empty` 信号量：初始化为缓冲区的大小 `BUFSIZE`。它表示缓冲区中还有多少个**空槽位**。
- `full` 信号量：初始化为0。它表示缓冲区中已经有多少个**被填充的槽位**。

生产者逻辑：

1. `sem_wait(&empty)`：想生产东西？先看看有没有空位。如果没有（`empty`**为0**），就乖乖等着。
2. `put()`：成功等到空位后，把产品放进缓冲区。
3. `sem_post(&full)`：放好东西后，通知消费者，“嘿，现在有东西可以取了！”。

消费者逻辑：

1. `sem_wait(&full)`：想取东西？先看看有没有产品。如果没有（`full`为0），就等着。
2. `get()`：成功等到产品后，从缓冲区取走。
3. `sem_post(&empty)`：取走东西后，通知生产者，“嘿，现在有空位可以放东西了！”。

遗留问题：多个生产者/消费者怎么办？

上面的方案在单个生产者和单个消费者时工作得很好。但如果**有多个生产者**（或多个消费者），就会出现新的竞态条件。比如两个生产者可能同时更新生产指针 `ppos`，导致数据被覆盖。

解决方案：在访问共享缓冲区（如 `ppos` 和 `cpos` 指针）的代码区域，额外加一个**互斥锁**来保护。这样就确保了任何时候只有一个生产者在修改 `ppos`，也只有一个消费者在修改 `cpos`。

常见并发错误 (Concurrency Bugs)

编写并发程序很难，很容易出错。我们来看看几种最典型的错误。

1. 原子性错误 (Atomicity Violation)

定义：一段代码逻辑上需要被当作一个整体执行，不可分割（即原子操作），但实际上却被其他线程中途打断了。

例子：

线程T1检查 if (shared > 23)，然后打印 printf("...%d", shared)。如果在 if 判断和 printf 执行之间，线程T2把 shared 的值改成了12，那么T1打印出的信息就会是 "Shared is >23: 12"，这显然是错误的。

修复：使用锁来保护需要原子执行的代码块。在T1中，`lock()` 应该在 `if` 之前，`unlock()` 应该在 `printf` 之后。

2. 顺序错误 (Order Violation)

定义：代码的正确性依赖于两个线程中操作的执行顺序，但却没有机制来强制保证这个顺序。

例子：

线程1创建了一个新线程（线程2），然后立刻去设置新线程的一个状态变量 mThread->State。但线程2可能在线程1完成状态设置之前就开始运行，并试图读取这个还未初始化的状态 mThread->State，导致程序出错。

修复：使用条件变量或信号量来强制执行顺序。线程1完成初始化后，发一个 `signal`，线程2在开始工作前，先 `wait` 这个信号。

3. 死锁 (Deadlock)

定义：两个或多个线程相互等待对方释放资源，导致所有线程都被卡住，程序无法继续前进。

例子：

- 线程1：先锁 `L1`，再锁 `L2`。
- 线程2：先锁 `L2`，再锁 `L1`。

如果线程1锁住了 `L1` 后被中断，然后线程2运行并锁住了 `L2`。这时，线程1想去锁 `L2`（已被线程2持有），线程2想去锁 `L1`（已被线程1持有），两个线程就陷入了无限的相互等待中，这就是死锁。

修复：最常见的策略是**规定一个全局的锁获取顺序**。比如，可以按锁变量的内存地址从小到大来获取。所有线程都必须遵守这个规则，这样就不会产生循环等待了。

Chapter 36-38

操作系统如何与设备交互

1. 演进之路：从混乱到有序

在很久以前，每个应用程序都需要自己编写直接与硬件设备沟通的代码。想象一下，你的代码里不仅要包含业务逻辑，还得知道怎么操作某个特定型号的打印机，这简直是一场噩梦！每个设备的接口都是独一无二的，而且应用程序通过一种叫做“轮询”的方式，不断地去问设备“你好了没有？”，非常低效。

现代操作系统改变了这一切。它接管了所有设备的管理工作，为应用程序提供了一套**统一、抽象的接口**。你只需要调用 `read()` 或 `write()` 这样的标准函数，剩下的复杂工作全由操作系统和**设备驱动程序**来完成。这种方式的核心是一种更高效的通信模式：**中断驱动**。

2. 核心架构：北桥与南桥

为了理解操作系统如何与硬件通信，我们需要了解一点计算机的主板架构。传统上，主板上有两个核心芯片组，我们称之为**北桥**和**南桥**。

- **北桥 (Northbridge)**：可以看作是“高速公路”的枢纽。它直接连接 CPU、内存（RAM）和高速显卡（GPU）这些需要极高数据传输速率的组件。
- **南桥 (Southbridge)**：负责连接那些相对较慢的设备，比如硬盘、USB 端口、网卡等。南桥并不直接与 CPU 通信，而是通过北桥这个“中介”来传递信息。

这种分层结构很好地隔离了高速和低速设备，让系统整体运行更加高效。

3. 标准通信协议：轮询、中断与 DMA

操作系统与设备之间的通信遵循一套标准流程。我们可以把每个设备想象成一个有三个“信箱”的黑盒子：一个**状态寄存器**（显示“忙”或“闲”）、一个**命令寄存器**（告诉它做什么）和一个**数据寄存器**（传递数据）。

- **轮询 (Polling) / 编程IO (PIO)**：这是最基本但效率最低的方式。
  1. CPU 不停地检查设备的状态寄存器，直到它变为“闲置”。（**忙等待**）
  2. CPU 将要发送的数据逐字节写入数据寄存器。
  3. CPU 将命令写入命令寄存器，让设备开始工作。
  4. CPU 再次不停地检查状态寄存器，直到设备完成任务。（**忙等待**） 这种方式的致命缺点是 CPU 在“忙等待”期间被完全占用，无法处理其他任何事情。
- **中断 (Interrupts)**：为了解放 CPU，我们引入了中断机制。
  - CPU 向设备发出命令后，不再等待，而是立即切换去执行其他任务。
  - 当设备完成任务后，它会主动向 CPU 发送一个**中断信号**。
  - CPU 收到信号后，暂停当前工作，处理中断（比如读取设备返回的数据），然后再切回原来的任务。
  - 这种异步方式极大地提高了 CPU 的利用率，实现了计算和 IO 的**重叠执行**。
  - **注意**：中断并非万能。在极高 IO 负载下（比如网络风暴），频繁的中断处理本身也会耗尽 CPU 资源，导致系统虽然在不停地响应中断，却无法处理真正的应用程序任务，这种现象被称为**活锁 (Livelock)**。因此，现代系统通常是轮询和中断的混合使用。
- **直接内存访问 (Direct Memory Access, DMA)**：对于大量数据的传输，如果还让 CPU 逐字节地搬运，效率依然很低。DMA 就是为此而生的终极优化。
  - CPU 告诉 DMA 控制器：“请把内存地址 A 处的 1MB 数据，直接传送到设备 B”。
  - 之后，DMA 控制器会全权负责数据的传输，整个过程无需 CPU 干预。
  - 传输完成后，DMA 控制器通过一次中断通知 CPU “任务完成”。
  - DMA 进一步解放了 CPU，让它能专注于计算任务。

设备驱动程序 (Device Drivers)

我们知道操作系统提供统一的接口，但市面上有成千上万种设备，它们的具体协议千差万别。**设备驱动程序**就是填补这个鸿沟的桥梁。

驱动程序是专门为某个特定设备或一类设备编写的内核代码。它的作用是：

- **对下**：与具体的硬件设备打交道，处理该设备独特的命令、状态和数据格式。
- **对上**：向操作系统的其余部分提供一个标准的、抽象的接口（例如，块设备接口、网络设备接口）。

这正是**封装**思想的完美体现。操作系统不需要知道每款硬盘的具体实现细节，它只需要通过统一的块设备接口与硬盘驱动程序交互即可。

深入硬盘 (HDD)

硬盘是数据持久化的核心。理解它的工作原理对于优化性能至关重要。

1. 硬盘的物理结构与耗时

一个机械硬盘（HDD）主要由旋转的**盘片 (Platter)** 和可移动的**磁头 (Head)** 组成。数据被存储在盘片上的一系列同心圆环（**磁道 Track**）中，每个磁道又被划分为若干个**扇区 (Sector)**。

一次硬盘 IO 操作的时间由三部分构成：

1. **寻道时间 (Seek Time)**：移动磁头到目标磁道所需的时间。这是**最耗时**的部分，通常需要几毫秒到十几毫秒。
2. **旋转延迟 (Rotational Latency)**：等待盘片旋转，直到目标扇区的起始位置转到磁头下方所需的时间。对于一块 7200 RPM (每分钟转速) 的硬盘，转一圈大约需要 8.3 毫秒。
3. **传输时间 (Transfer Time)**：磁头从扇区上读取或写入数据所需的时间。这部分非常快，通常在微秒级别。

结论很明确：**硬盘 IO 的性能瓶颈在于寻道和旋转**。因此，**顺序 IO**（连续读取相邻的块）的性能远高于**随机 IO**（在盘片上到处跳跃着读取块）。

2. IO 调度算法

既然寻道时间是主要开销，操作系统就需要智能地安排待处理的 IO 请求队列，以最小化磁头的移动距离。这就是 **IO 调度**。

- **先来先服务 (FCFS)**：最公平但通常效率最低。如果请求在盘片上随机分布，磁头会像疯了一样来回移动。
- **最短寻道时间优先 (SSTF)**：优先处理离当前磁头位置最近的请求。这能显著减少寻道时间，但可能会导致离得远的请求长时间得不到服务，即**饥饿 (Starvation)**。
- **电梯算法 (SCAN / Elevator)**：磁头在一个方向上（例如从内到外）移动，处理沿途的所有请求。到达边缘后，再反向移动。这种算法既保证了效率，又避免了饥饿，非常常用。

超越单个磁盘：RAID

单个磁盘总有其局限性：容量有限、性能有限，而且一旦损坏数据就全没了。为了解决这些问题，我们引入了一个新的抽象层：**RAID (廉价冗余磁盘阵列)**。

RAID 的核心思想是用多个物理磁盘组合成一个逻辑上更大的、更快的、更可靠的“虚拟磁盘”。

- **RAID 0 (条带化)**：将数据块交错地写入多个磁盘。比如块1写在磁盘0，块2写在磁盘1，块3写在磁盘0..... 这能极大地提升读写性能，因为多个磁盘可以并行工作。但它没有任何冗余，任何一块磁盘损坏，所有数据都会丢失。
- **RAID 1 (镜像)**：将同样的数据完整地写入两块磁盘。这提供了极高的数据可靠性，一块磁盘损坏，另一块上还有完整的备份。但它的磁盘利用率只有 50%。
- **RAID 5 (分布式奇偶校验)**：这是一种很好的折中方案。它像 RAID 0 一样做条带化，但同时会将一个叫做“奇偶校验”的冗余信息也分布存储在所有磁盘上。这使得阵列可以容忍**任意一块磁盘损坏**，同时磁盘利用率也比较高。
- **RAID 10 (1+0)**：这是一种组合级别，先将磁盘两两配对做成 RAID 1 (镜像)，然后再将这些镜像对组合起来做成 RAID 0 (条带化)。它兼具了 RAID 1 的高可靠性和 RAID 0 的高性能，在企业级应用中非常受欢迎。

Chapter 39-40

文件系统的目的与抽象

首先，文件系统的根本任务是什么？它面对的是一堆持久化的存储块，比如硬盘上的扇区。文件系统的目标就是有效地管理这些块，并为用户和程序提供一个有结构、有组织的视图。它需要解决几个核心问题：如何组织数据、谁有权访问、如何共享和并发访问文件。

为了实现这个目标，文件系统提供了两个核心的抽象：

- **文件 (File)**：在用户看来，文件是一个线性的、持久的字节数组。你可以对它进行读写操作。
- **目录 (Directory)**：它本身也是一种特殊的文件，其内容是一个列表，包含了从文件名到其他文件或子目录的映射。

文件系统的层次结构

为了将简单的硬件块变成我们熟悉的文件和目录，操作系统内部构建了一个层次化的结构。从下到上依次是：

1. **磁盘硬件 (Disk Hardware)**：物理设备本身。
2. **磁盘设备驱动 (Disk Device Drivers)**：与硬件直接通信的底层软件。
3. **文件访问 (File Access)**：这一层关注文件的具体操作，比如创建、删除、读、写等。
4. **目录访问 (Directory Access)**：负责解析路径，将文件名字符串映射到具体的文件对象。
5. **文件命名 (File Naming)**：最顶层，定义了文件的命名规则和语法，是用户直接能感知到的部分。

现代操作系统大多采用**无类型文件**的理念，即文件系统本身不关心文件里存的是什么内容，一律看作是字节序列。文件的“类型”通常由应用程序通过文件扩展名（如 `.txt`，`.jpg`）来解释。

文件的三种视图

要真正理解文件系统，最关键的是要明白，对于同一个文件，系统中的不同角色有完全不同的“看法”。一共有三种视图。

1. 用户的视图：文件名 (Filename)

这是我们最熟悉的视图。我们通过一个人类可读的路径（比如 `/home/sanidhya/11-proc.md`）来识别和访问文件。这种层次化的命名方式非常直观，但计算机内部并不直接使用它。

- 目录的作用就是将这些**文件名**映射到文件在操作系统内部的唯一标识。
- 当你访问一个路径时，文件系统会逐级解析目录，例如访问 `/tmp/test.txt` 会先查找根目录 `/` 找到 `tmp` 目录，再在 `tmp` 目录中找到 `test.txt` 文件。
- 每个目录中都有两个特殊的条目：`.` 指向当前目录，`..` 指向上级目录。

2. 操作系统的视图：Inode 和设备 ID

在文件系统内部，每个文件和目录都有一个唯一的“身份证”，这个身份证就是 **inode (索引节点)**。

- 一个 inode 是存储在磁盘上的一块元数据 (metadata)。它包含了除了文件名之外的几乎所有文件信息：文件大小、权限、所有者、创建时间，以及最重要的——指向存储文件内容的**数据块指针**。
- 在同一个文件系统中，每个文件的 inode 编号都是唯一的。操作系统通过这个 inode 号来精确地定位文件。
- **硬链接 (Hard Link)**：**允许多个文件名指向同一个 inode。这也是为什么 inode 本身不存储文件名的原因，因为一个文件可以有多个名字。inode 中有一个“引用计数”，记录有多少个文件名指向它。**

3. 进程的视图：文件描述符 (File Descriptor)

进程在和文件交互时，如果每次都使用长长的文件名字符串，效率会非常低下，因为操作系统需要反复遍历目录树来查找对应的 inode。

为了解决这个问题，引入了**文件描述符 (File Descriptor, fd)**。

- 当一个进程调用 `open()` 打开一个文件时，操作系统会进行一次从文件名到 inode 的查找。
- 然后，操作系统会在该进程内部维护的一个**打开文件表**中创建一个新条目。这个条目包含了指向文件 inode 的指针以及其他与该进程相关的信息，比如当前读写的位置（偏移量）。
- `open()` 系统调用最终返回给进程的，不是 inode 号，而是这个新条目在表中的**索引**——一个很小的非负整数（例如 3, 4, 5...）。这个整数就是**文件描述符**。
- 之后，进程进行 `read()`, `write()` 等操作时，只需传入这个高效的文件描述符 `fd` 即可。

核心文件系统 API

我们日常使用的文件操作，最终都会转化为以下这些核心的系统调用：

- **文件操作**：`create`, `open`, `close`, `read`, `write`, `seek`。
- **目录操作**：`createdir` (或 `mkdir`), `rmdir`, `readdir`。
- **删除文件**：`unlink`。这个调用很有意思，它并不是直接删除文件数据。它只是从目录中移除一个文件名条目，并将对应 inode 的引用计数减一。只有当引用计数变为 0 时（即没有任何文件名指向这个 inode），文件系统才会回收该 inode 和其占用的数据块。

管理多个文件系统

一个现代操作系统通常需要同时处理多个文件系统，比如来自不同硬盘分区、U盘或者网络存储。对此，主要有两种组织方式：

- **Windows 方式**：为每个文件系统分配一个盘符（如 `A:`, `C:`），每个盘符都是一个独立的根目录。
- **Unix/Linux 方式**：将所有文件系统组织成一个单一的、统一的目录树。一个主文件系统作为根 `/`，其他文件系统可以被“挂载 (mount)”到这个树的任意目录节点上。例如，`/home` 目录下的所有文件可能物理上存储在另一块独立的硬盘上。

Chapter 41-42

文件系统实现的挑战

我们面临的核心问题是：如何在一块大容量磁盘（本质上是一个巨大的块数组）上，用高效的数据结构来组织和存储文件与目录。设计时必须考虑几个关键因素：

- **空间开销**：用于管理文件（即元数据）的数据结构本身不应占用太多空间。
- **碎片问题**：要尽量减少内部碎片（块内浪费的空间）和外部碎片（空闲块不连续导致大文件无法分配）。
- **访问效率**：无论是顺序读取整个文件，还是随机跳到文件中间某个位置，访问速度都必须很快。

为了应对这些挑战，不同的文件系统（如 EXT4, NTFS, FAT）采用了不同的策略。而操作系统为了能够同时支持这些不同的文件系统，引入了一个叫做**虚拟文件系统 (Virtual File System, VFS)** 的抽象层。VFS 定义了一套通用的文件系统接口，所有具体的文件系统都去实现这套接口，这样操作系统内核就可以用统一的方式与它们交互了。

文件数据块的分配策略

一个文件的数据可能很大，需要占用多个磁盘块。如何组织这些块是文件系统实现的核心。我们来看看几种主要的分配策略，分析它们的优缺点。

1. 连续分配 (Continuous Allocation)

这是最简单直接的方法：将一个文件的所有数据块，连续地存放在磁盘上。

- **优点**：顺序读性能极好，因为磁头不需要移动，只需连续读取即可。元数据开销极小，只需在 inode 中**记录起始块号和文件长度**。
- **缺点**：存在严重的**外部碎片**问题，用久了磁盘上会有很多小块的空闲空间，但找不到足够大的连续空间来存放新文件。并且，文件一旦创建就很难增长。
- **适用场景**：主要用于只读介质，如 CD/DVD，因为上面的文件不会改变。

2. 链接分配 (Linked Allocation)

为了解决连续分配的缺点，链接分配将文件块像链表一样串联起来。**每个数据块的末尾都包含一个指针，指向文件的下一个数据块。**

- **优点**：彻底消除了外部碎片，文件可以轻松地动态增长。
- **缺点**：**随机访问性能极差。**要想到达文件的第 N 个块，必须从头开始，沿着指针链逐块访问 N-1 次。

- **文件分配表 (FAT)**：这是链接分配的一个著名变种。它将所有块的“指针”集中存放在磁盘开头的一个大表里，这个表就是文件分配表 (File Allocation Table)。这样随机访问的性能有所改善，因为可以先在内存中查表，但这个表本身也可能成为瓶颈。

3. 索引分配 (Indexed Allocation)

这种方法在每个文件的 inode 中，直接存放一个**指针数组**，数组中的每个指针分别指向文件的一个数据块。

- **优点**：**随机访问性能极佳**。想访问哪个块，直接从 inode 的指针数组中拿到地址即可。文件也可以动态增长（只要数组里还有空指针）。
- **缺点**：对于小文件来说，inode 中大量的指针位都浪费了；对于大文件来说，inode 中有限的指针数量又限制了文件的最大尺寸。

4. 多级索引 (Multi-level Index)

为了结合索引分配的优点并克服其缺点，现代文件系统（如 Linux 的 ext4）普遍采用**多级索引**。这是一种非常聪明的混合策略。在一个 inode 中，指针被分为几类：

- **直接指针 (Direct Pointers)**：inode 中有十几个指针直接指向数据块。这使得小文件的访问非常快，只需一次磁盘读取（读 inode）就能找到所有数据。
- **一级间接指针 (Single Indirect Pointer)**：这个指针指向的不是数据块，而是一个“指针块”，这个块里面装满了指向数据块的指针。这让文件可以变得更大。
- **二级间接指针 (Double Indirect Pointer)**：指向一个“二级指针块”，该块里的每个指针又指向一个“一级指针块”。
- **三级间接指针 (Triple Indirect Pointer)**：以此类推，可以支持非常巨大的文件。

这种结构非常灵活，对小文件友好，又能支持海量数据，同时保持了高效的随机访问能力。

磁盘上的整体布局

一个典型的 Unix-like 文件系统在格式化后，磁盘空间会被划分为几个主要区域：

- **超级块 (Superblock)**：整个文件系统的“全局元数据”，存储在这个分区的固定位置。它记录了文件系统的类型、块大小、inode 和数据块的总数、空闲数量等关键信息。操作系统挂载文件系统时，首先就要读取它。
- **分配位图 (Allocation Bitmaps)**：通常有两个位图，一个用于 **inode**，一个用于**数据块**。每个 bit 对应一个 inode 或数据块，`1` 表示已分配，`0` 表示空闲。当需要创建文件或写入数据时，文件系统通过查找位图来快速找到可用的资源。
- **Inode 表 (Inode Table)**：一块连续的区域，用来存放所有的 inode 结构。
- **数据区 (Data Region)**：磁盘上剩余的大部分空间，用来存放真正的文件和目录内容。

文件操作的内部流程

了解了磁盘布局后，我们就能理解一个简单的文件操作（比如读取 `/foo/bar`）在底层需要多少次磁盘 IO 了：

1. **读取根目录 `/` 的 inode**，找到其数据块的位置。
2. **读取根目录的数据块**，在其中查找名为 `foo` 的条目，从而获得 `foo` 的 inode 号。
3. **读取 `foo` 的 inode**，找到其数据块的位置（因为 `foo` 是一个目录）。
4. **读取 `foo` 的数据块**，在其中查找名为 `bar` 的条目，从而获得 `bar` 的 inode 号。

5. **读取 `bar` 的 inode**，找到 `bar` 的元数据和指向其内容的数据块指针。
6. **读取 `bar` 的数据块**，将文件内容返回给用户。

可以看到，仅仅是一次简单的读取，就需要多次磁盘 IO。这也解释了为什么**文件描述符**和各种**缓存**（比如目录项缓存 `dcache`）对于文件系统性能如此重要，它们可以避免重复、昂贵的路径遍历。而像 `close()` 这样的操作，通常只是在进程内部释放文件描述符，几乎不涉及磁盘 IO。

崩溃的挑战

想象一下，你正在向一个文件追加内容。我们知道这个操作需要更新磁盘上的多个位置：

1. 写入新的**数据块**。
2. 更新**数据位图**，标记该数据块已被使用。
3. 更新文件的 **inode**，增加文件大小并添加指向新数据块的指针。

这些写入操作不是瞬时完成的。如果在这三次写入的中间，系统突然崩溃了（比如断电），会发生什么？磁盘上的数据结构就会进入一种不一致的“中间状态”，这可能导致严重的问题。

不一致的场景分析

我们来分析一下，如果在更新上述三个结构（inode, 数据位图, 数据块）的过程中发生崩溃，可能出现哪些不一致的情况：

- **只写了数据块**：新数据虽然在磁盘上，但没有任何元数据指向它。这块数据成了无主垃圾，下次会被覆盖。文件本身没有变化，数据丢失了。
- **只写了位图**：位图显示某个块被占用了，但实际上没有任何 inode 指向它。这个块就这么“泄露”了，文件系统会认为它已使用，但永远不会再访问它，造成空间浪费。
- **只写了 inode**：这是最糟糕的情况。Inode 现在指向了一个新的数据块，但这个块的内容可能是旧的垃圾数据，甚至可能属于另一个文件。此时读取文件就会读到错误的内容。
- **写了部分组合**：比如写了 inode 和位图，但没写数据。或者写了数据和位图，但没写 inode。这些都会导致各种各样的数据不一致问题。

核心问题在于，多个独立的写入操作，逻辑上需要成为一个**原子操作**，但物理上却无法保证。

传统解决方案：文件系统检查器 (fsck)

为了解决崩溃后的不一致问题，早期文件系统依赖一个叫做 `fsck` (File System Check) 的工具。

当系统在一次不正常关机后重启时，就会自动运行 `fsck`。它的工作方式是**扫描整个磁盘**，交叉验证所有的元数据，找出并尝试修复不一致的地方。比如：

- **检查链接数**：如果一个 inode 的链接计数是 1，但在所有目录中都找不到指向它的条目，`fsck` 会认为它是一个“孤儿”文件，并把它放到一个特殊的 `lost+found` 目录里，让你决定如何处理。
- **检查位图**：如果一个 inode 指向了某个数据块，但位图中却显示该块是空闲的，`fsck` 就会修复位图，**将对应位置为“已使用”**。
- **检查重复指针**：如果发现有两个不同的 inode 指向了同一个数据块，这是严重错误。`fsck` 可能会尝试复制这个数据块，让每个 inode 都有自己的副本。

fsck 的巨大缺陷

虽然 `fsck` 能解决一些问题，但它有两个致命的缺点：

1. **速度极慢**：对于现代TB级别的硬盘，完整扫描一次需要数小时甚至更久。在这期间，文件系统是不可用的，这对于服务器等应用是无法接受的。
2. **无法保证正确性**：`fsck` 只能尽力恢复文件系统的**一致性**，但无法恢复数据的**正确性**。它只能做“最佳猜测”，但有时它的修复操作可能会让事情变得更糟。

现代解决方案：日志 (Journaling)

由于 `fsck` 的缺陷，现代文件系统几乎都采用了一种更先进的技术来保证一致性，那就是**日志 (Journaling)**，也叫**日志记录**。

它的核心思想借鉴了数据库的**预写式日志 (Write-Ahead Logging)**。在对文件系统进行任何实际的修改之前，先把“要做什么”这件事，作为一个整体的事务（transaction），原封不动地写入磁盘上一个特殊的、连续的日志区域（Journal）。

一个典型的日志文件系统操作流程如下：

1. **日志写入 (Journal Write)**：当要追加一个文件时，文件系统首先把所有要做的修改（比如：[写数据到块D]、[更新位图B]、[更新inode I]）打包成一个事务，并写入到日志区。
2. **日志提交 (Journal Commit)**：当整个事务都写入日志后，再向日志区写入一个特殊的“提交记录”。这一步是强制写入磁盘的。到此为止，所有实际的文件系统结构都还没有被修改。
3. **检查点 (Checkpoint)**：现在，文件系统才开始将日志中的变更，逐一写入到它们在磁盘上最终的位置（比如更新真正的 inode 和位图）。
4. **释放日志空间**：当所有变更都安全地写入最终位置后，日志中对应的这条事务记录就可以被标记为无效，空间可以被回收。

日志如何保证一致性？

- **崩溃发生在第1步**：日志里只有不完整的事务，没有提交记录。重启后，系统直接忽略这条不完整的日志，文件系统保持原样，什么都没发生。
- **崩溃发生在第2步或第3步**：日志里有一条完整的、已提交的事务。重启后，系统会检查日志，发现这个事务已经提交但可能没有完成最终写入。于是，系统会**重做 (redo)** 这个事务，把日志里记录的变更再次写入到文件系统的最终位置。
- **崩溃发生在第4步**：没关系，所有数据已经安全写入，只是日志没来得及清理，下次清理即可。

通过这种方式，文件系统的更新操作变成了**原子性**的。它要么完全成功，要么就像从未发生过一样，永远不会停留在不一致的中间状态。最关键的是，恢复过程非常快，只需检查日志文件即可，不再需要扫描整个磁盘。这就是为什么现在的电脑即使意外断电，重启后也能很快恢复正常。